Bachelor thesis at Ulm University of Applied Sciences
Department of Computer Science

Degree Program **Informatik**

Presented by
Gabriel Zimmermann
April 2026

1. Reviewer: **Prof. Dr.-Ing. Klaus Baer**
2. Reviewer: **Prof. Dr.-Ing. Philipp Graf**

Place, Date, Signature

Diese Abschlussarbeit wurde von mir selbständig verfasst. Es wurden nur die angegebenen Quellen und Hilfsmittel verwendet. Alle wörtlichen und sinngemäßen Zitate sind in dieser Arbeit als solche kenntlich gemacht.

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Place, Date, Signature

Plugin Architectures with Rust - an Analysis of Obstacles and Prospects

Gabriel Zimmermann

Abstract

Plugin architectures enable extensible software systems by allowing runtime loading of external modules. While Rust offers memory safety guarantees and performance characteristics ideal for plugin development, the language's lack of a stable Application Binary Interface (ABI) presents significant challenges for cross-module compatibility. This thesis examines the landscape of plugin system implementations in Rust, evaluating three primary approaches: native Rust plugins, C ABI-compatible interfaces, and Inter-Process Communication (IPC) mechanisms. Through comparative analysis of the performance overhead, development complexities, language limitations and interoperability that arise through the implementations of plugins, this work demonstrates that Rust's unstable ABI hampers reliable direct plugin loading across compiler versions. The findings indicate that C ABI remains the optimal choice when performance is paramount, while IPC-based architectures provide superior isolation and version compatibility at the cost of increased overhead. These results contribute practical guidance for developers designing extensible Rust applications and highlight ongoing gaps in Rust's ecosystem for safe, performant plugin architectures.

I like to thank my supervisor Michael Henn for his guidance and support throughout this project. His knowledge of the Rust ecosystem and perspective from his work experiences with Rust were invaluable in shaping the direction of this thesis and providing feedback on the analysis.

I also want to thank the Rust community at large for their open discussions and resources around plugin systems, which provided much of the background context for this work.

Finally, I am grateful to my friends and family for their encouragement and understanding during the long hours spent researching and writing this thesis.

Contents

Abbreviations:

- ABI: Application Binary Interface
- IPC: Interprocess Communication
- RPC: Remote Procedure Call
- FFI: Foreign Function Interface

I. Introduction

Modern software systems increasingly demand flexibility and extensibility. Plugin architectures have emerged as a fundamental pattern for achieving this goal, enabling applications to load additional functionality at runtime without recompilation. As highlighted by Apple's technical documentation, "Plug-in architectures are an attractive solution for developers seeking to build applications that are modular, customizable, and easily extensible" [1].

The rise of the rust language popularity has led to an increased interest in using Rust for plugin development. Rust has established itself as a powerful systems programming language that offers memory safety guarantees and performance characteristics. However, the language's design philosophy prioritizes compile-time guarantees over binary stability, resulting in an intentionally unstable Application Binary Interface (ABI). Small changes in your written code, can impact the resulting binary in a way that it is not compatible with the previous version. With that being said, we can circumvent this problem by using other approaches.

One popular approach is to use C ABI-compatible interfaces, which can provide a stable interface for plugins while still allowing developers to write their plugins in Rust. The C ABI has been the backbone for many plugin systems in various programming languages, including the Rust ecosystem. But this approach requires the careful handling of unsafe code and can lead to some performance overhead due to the need for FFI (Foreign Function Interface) calls. In addition, this can lead to the loss of Rust's safety guarantees, as the C ABI does not provide the same level of safety as Rust's native interfaces.

Another approach is to use Inter-Process Communication (IPC) mechanisms, which can provide a more flexible and scalable architecture for plugin systems. However, this approach can also introduce additional complexity and overhead, as it requires the management of separate processes and communication channels. This will impact the performance of the plugin system, as IPC can introduce latency and reduce the overall efficiency of the system. However, it can provide better isolation and security for plugins, as they run in separate processes.

In this thesis, we are interested in understanding how different approaches within the Rust Ecosystem try to solve this problem, and therefore we want to evaluate them based on the performance overhead using a simple benchmark, the resulting development complexity, language limitations and interoperability.

II. Related work

We start by discussing the foundational concepts that underpin plugin systems and their relevance to Rust. This includes an exploration of what plugin systems are, how they enable extensibility in software applications, and the mechanisms through which plugins communicate with their host applications. Then, we delve into current state of the art in Rust plugin systems, highlighting the challenges posed by Rust's unstable ABI and the various approaches that have been proposed or implemented to address these challenges.

A. Foundational Concepts

To contextualize the challenges of plugin systems in Rust, it is essential to establish a few definitions, ranging from the concept of plugins in software design to how developers can communicate with other pieces of software.

a) *Plugin Systems:*

A plugin is a "computer software that adds new functions to a host program without altering the host program itself" [2]. A plugin system is a software architecture pattern that allows a host application to be extended by third-party modules (plugins) without modifying the host's source code or recompiling the entire system. Key characteristics include extensibility, modularity, and dynamic loading capabilities.

- **Extensibility**: The ability to add new functionality post-deployment.
- **Modularity**: Plugins are isolated units that interact with the host through a well-defined interface.
- **Dynamic Loading**: Plugins are typically loaded at runtime rather than linked statically at compile time.

One typical way to visualize plugin systems is through the metaphor of a jigsaw puzzle, where the host application provides one or more interfaces and the plugins are pieces that fit into these interfaces to extend functionality .

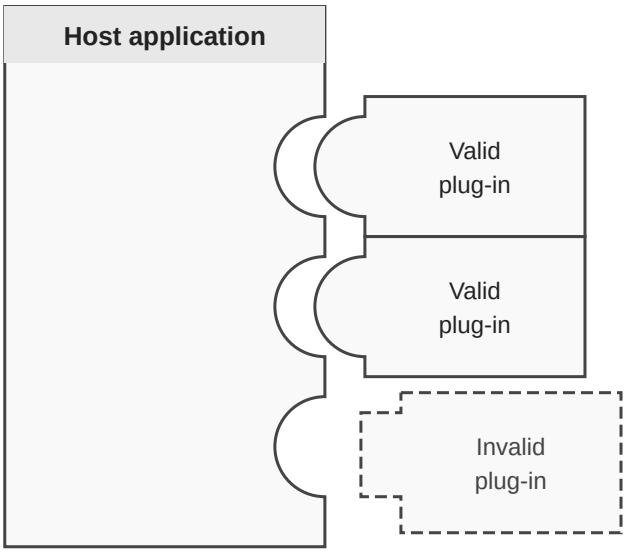

Fig. 1. Illustration between host application and plugins and their connections by the Apple developers

When designing a plugin system, developers must consider how the host and plugins will communicate, which leads to the concept of dynamic linking - using an Application Binary Interface (ABI) - or inter-process communication (IPC).

When writing a Plugin systems, developers generally follow one of two patterns:

- **Static Linking**: Plugins are compiled into the host binary. This offers performance benefits but lacks runtime flexibility.

- **Dynamic Linking**: Plugins are separate binary files loaded into the host process memory space at runtime.

In this thesis, the focus is on dynamic linking, as it necessitates strict ABI compatibility or robust isolation mechanisms.

b) *Application Binary Interface (ABI):*

While the Application Programming Interface (API) defines a "sets of standardized requests that allow different computer programs to communicate with each other" [3] like function names and argument types, the Application Binary Interface (ABI) operates at a lower level, governing the interaction between compiled machine code.

This covers aspects such as, illustrated in Fig. 2:

- **Calling Conventions**: How function arguments are passed (registers vs. stack), how return values are delivered, and who is responsible for cleaning up the stack (caller vs. callee).
- **Data Layout**: The memory alignment, padding, and byte-ordering (endianness) of data structures.
- **Name Mangling**: The algorithm used by compilers to encode function names (including type information) into symbol names for the linker.
- **Exception Handling**: The mechanism for propagating errors or exceptions across module boundaries.

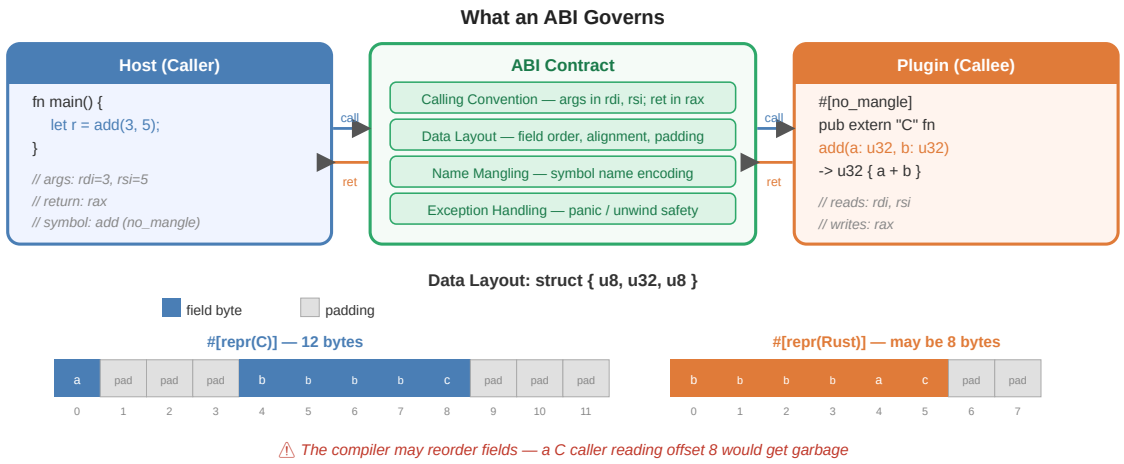

Fig. 2. The ABI as a contract between caller and callee: how a function call crosses the boundary (top), and how struct field layout differs between `#[repr(C)]` and `#[repr(Rust)]` (bottom).

Challenges in Rust: Dynamic linking in Rust is complicated by the language's ownership model and panic handling. Since Rust does not guarantee a stable ABI, a plugin compiled with Rust version 1.70 might use a different struct layout or calling convention than a host compiled with version 1.75. Furthermore, if a plugin panics, the unwinding behavior must be contained; otherwise, it can lead to undefined behavior or crashes in the host process. As described in implementation guides, "panics cannot cross the FFI boundary" without explicit handling, such as wrapping function bodies in `catch_unwind` or using ABI-stable libraries like `abi_stable` [4].

Foreign Function Interface (FFI): An FFI is the mechanism by which a program written in one language calls functions defined in another language. In practice this means crossing a compiled-language boundary — for example, Rust calling a C function, or C calling a Rust function. Because different languages may use different calling conventions, name-mangling schemes, and data representations, an FFI requires both sides to agree on a shared ABI. In Rust, this is expressed with `extern "C"` blocks and `#[no_mangle]` annotations, which instruct the compiler to use the stable C calling convention instead of Rust's own (unstable) one. The term "FFI boundary" therefore refers to the point in code where execution crosses from one language's runtime into another.

c) *Virtual Dispatch Tables (VTables):*

A **virtual dispatch table** (vtable) is a compiler-generated data structure that enables runtime polymorphism. When a Rust value is used through a `dyn Trait` reference — i.e., a trait object — the compiler does not know at compile time which concrete type implements the trait, so it cannot inline or statically resolve the method calls. Instead, it creates a fat pointer: a pair of (data pointer, vtable pointer). The vtable is a table of function pointers, one per method of the trait, pointing to the concrete implementations for that specific type.

For example, given a trait `Fuser` with a method `fuse`, a `Box<dyn Fuser>` holding a `ConcreteFuser` will carry a pointer to a vtable that contains the address of `ConcreteFuser::fuse`. Calling `fuser.fuse(...)` becomes an indirect call through the vtable, which incurs a small overhead compared to a direct (monomorphised) function call, but allows any type implementing `Fuser` to be used interchangeably at runtime.

Relevance to plugin systems: VTables are inherently tied to the ABI of the compiler that generated them. Because Rust does not guarantee a stable ABI, the layout of a vtable — the order and representation of function pointers — may differ between compiler versions or even between compilation units. This means that a `dyn Trait` object created in a plugin compiled with one version of the Rust compiler cannot safely be passed to a host compiled with a different version: the host would read function pointers from wrong offsets in the vtable, leading to undefined behavior. Additionally, if the shared library that contains the function implementations is unloaded while a vtable referencing it still exists, any subsequent call through that vtable will dereference dangling pointers. Crates such as `abi_stable` work around this by replacing Rust's native vtables with manually constructed, `#[repr(C)]`-stable equivalents whose layout is explicitly fixed.

d) *Dynamic Linking and Runtime Loading:*

Dynamic Linking is the process of resolving references to external libraries or modules at runtime, rather than at compile time (link time).

Mechanism: In Unix-like systems, this is achieved through the `dlopen` (dynamic open), `dlsym` (dynamic symbol lookup), and `dlclose` (dynamic close) system calls. In Windows, the equivalents are `LoadLibrary`, `GetProcAddress`, and `FreeLibrary`.

Process Flow, as illustrated in Fig. 3:

- **Discovery**: The host application locates the plugin binary file (e.g., .so, .dll, .dylib).
- **Loading**: The dynamic linker (on Linux: `ld-linux.so`) maps the plugin's code and data segments into the host's virtual memory space using the `mmap` system call. `mmap` (memory-map) instructs the kernel to project a region of a file directly into the process's virtual address space without copying it into a conventional buffer — the file's contents become accessible as ordinary memory pages, loaded on demand. `ld-linux.so` is the OS-provided program interpreter that is invoked automatically when an ELF binary is executed; it parses the binary's dependencies, locates the required shared libraries on disk, `mmap`s them into the process's address space, and patches all relocation entries so that symbol references point to the correct memory addresses before `main` is called.
- **Resolution**: The host resolves symbol addresses (function pointers) within the loaded plugin using `dlsym`.
- **Execution**: The host invokes the plugin's functions via the resolved pointers. Because all segments reside in the same address space, calls are direct function-pointer jumps with no serialization overhead.

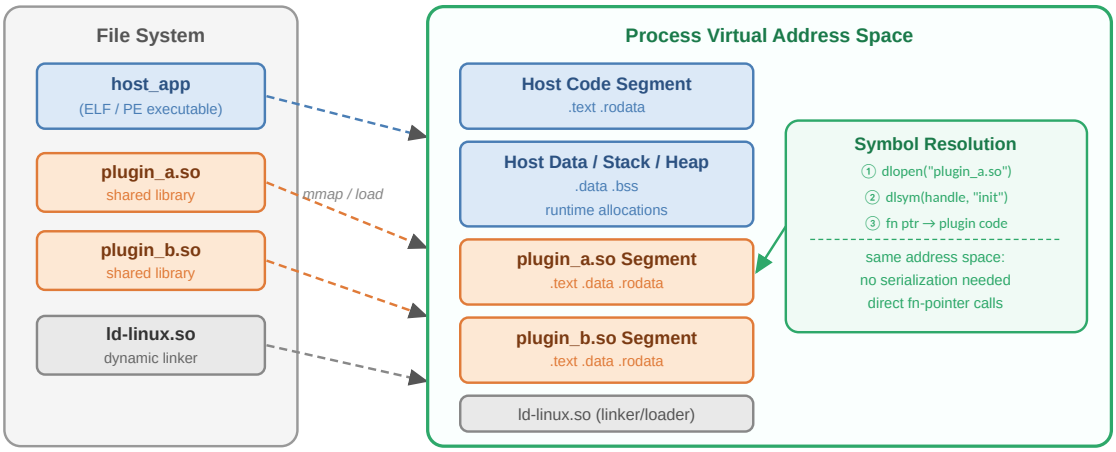

Fig. 3. How the dynamic linker maps shared libraries into the host process's virtual address space at runtime, enabling direct function-pointer calls without serialization.

In Rust, one crate has emerged regularly when loading dynamic libraries: `libloading` [5], which provides a safe abstraction over platform-specific dynamic loading APIs. However, using `libloading` still requires careful management of ABI compatibility and error handling, as it does not solve the underlying issues of Rust's unstable ABI or panic safety.,

e) *Inter-Process Communication (IPC):*

Inter-Process Communication (IPC) refers to the mechanisms that allow processes to exchange data and signals. Unlike dynamic linking, where a plugin is loaded directly into the host process's address space, IPC keeps the host and plugin in separate, isolated processes that communicate through an OS-managed channel, as illustrated in .

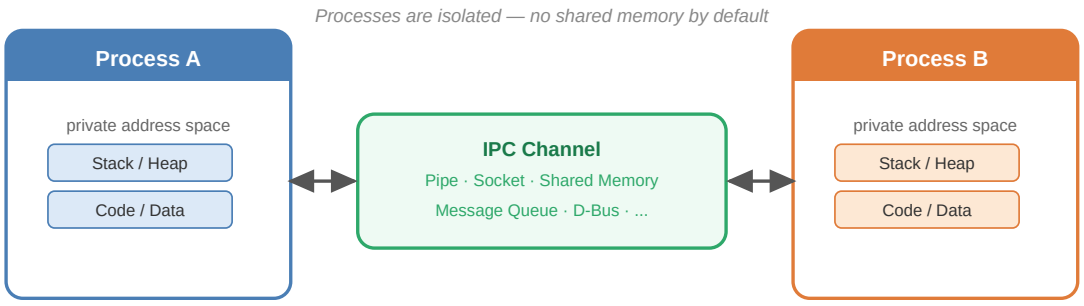

Fig. 4. Two isolated processes communicating through an IPC channel.

Common IPC methods include:

- **Shared Memory**: Multiple processes access the same memory region for communication, requiring synchronization mechanisms to prevent race conditions
- **Message Passing**: Processes send messages to each other through queues, pipes, or sockets, which can be synchronous or asynchronous
- **Remote Procedure Calls (RPC)**: A higher-level abstraction where a process can invoke a procedure in another process as if it were a local function call, often implemented over network protocols

Relevance to Rust: While IPC can be used to enable communication between a Rust host and plugins, it introduces additional complexity and overhead compared to in-process dynamic linking. Moreover, Rust's ownership model and safety guarantees can complicate the design of IPC mechanisms, as shared memory requires careful synchronization, and

message passing may involve serialization and deserialization of complex data structures, which can be error-prone and inefficient. As such, while IPC is a viable approach for plugin systems, it is often considered a last resort when ABI stability cannot be achieved through dynamic linking.

B. Existing work

As of 2026, the Rust ecosystem remains in a state of intentional ABI instability, a condition that has persisted despite years of community advocacy and formal proposals. Many proposals were held, such as RFC 1675 [6], RFC 600 [7] and discussion in the Rust Programming Language Forums, like “A Stable Modular ABI for Rust” [8], which proposes to enable runtime loading of Rust libraries without C interop. However, the proposal was ultimately deferred, with the compiler team citing the high cost of freezing internal representations that are critical for future optimizations, such as constant evaluation and monomorphization strategies [6]. Instead of a native solution, the community has converged on pragmatic workarounds, where the usage of the C ABI is the only reliable standard for binary interoperability, forcing developers to adopt verbose `extern "C"` interfaces or rely on third-party crates like abi_stable to simulate stability through `#[repr(C)]` wrappers and runtime checks [4], [9]. While experimental efforts continue to explore WebAssembly as a language-agnostic ABI alternative, no native Rust-to-Rust stable ABI has been implemented, leaving plugin developers to navigate a fragmented landscape of manual FFI management and abstraction layers.

No comprehensive comparison between possible solutions for plugin systems in Rust has been done yet. There are some simple comparisons, but they don’t cover the full range of options and trade-offs.

III. Methods

A. Analysis of the requirements

Goal of the research was to analyze the following points:

- Performance Overhead
- Development Complexities
- Language Limitations
- Safety
- Interoperability

In the following section we will introduce these points in more detail and explain how we will analyze them in the context of plugin systems in Rust.

a) *Performance Overhead:*

When measuring our plugins, we expect that the execution time of the plugin's internal functionality to be relatively similar across different approaches. However, the setup time for each approach is expected to vary, with approaches that require more complex initialization (e.g. loading a dynamic library, setting up IPC communication, serialization) are more likely to require more time.

We expect that the IPC approach will have higher performance overhead compared to the other approaches due to the need for serialization and deserialization of data, as well as the communication overhead between processes. This is because IPC typically involves more complex interactions between processes, which can introduce additional latency compared to approaches that operate within a single process. We expect the measurements for the Rust ABI and C ABI approaches to be more similar to each other, as they both involve loading and executing code within the same process, albeit with different calling conventions and potential overhead from FFI in the case of the C ABI approach.

To measure the results, we are using the **criterion** and **gungraun** crate for our benchmarks. The results of these will be separated into setup time and function execution time, separated into "single fuse" (1 array element) and "multi fuse" (1,000,000 elements).

The separation for the function execution time into "single fuse" and "multi fuse" is important, as it allows us to understand how the performance of each approach scales with the number of sensors. The "single fuse" case will give us insight into the overhead of each approach when dealing with a small amount of data, while the "multi fuse" case will show us how well each approach handles larger amounts of data and whether there are any performance bottlenecks that arise as the number of sensors increases.

As noted by the criterion documentation, the criterion crate is a statistics-driven micro-benchmarking library, which aims to provide strong statistical confidence in detecting and estimating the size of performance improvements and regressions [10]. From the gungraun documentation we can gather, that "gungraun is a one-shot benchmarking harness and framework which uses Valgrind's Callgrind, Cachegrind, and DHAT to provide extremely accurate and consistent measurements of Rust code, making it perfectly suited to run in environments like a CI" [11]. Here we are in particular interested in the amount of instructions and the estimated execution cycles. It is important to understand, that not each instruction requires the same amount of execution cycles, and that the number of instructions is not necessarily a good indicator for the performance of a particular approach.

We hope by combining both of these tools, we can get a comprehensive understanding of the performance characteristics of each approach, and how they compare to each other in terms of setup time and function execution time. If the instruction count is notably higher than other approaches, but uses the same amount of execution cycles, and likely a similar amount of time, then this is not necessarily a bad result for the approach. However, if the instruction count is notably higher and also uses more execution cycles, then this is likely a bad result for the approach.

In Rust benchmarking, wrapping inputs and intermediate values in `std::hint::black_box` is essential to prevent the compiler from optimizing away the

code you intend to measure. Because Rust's optimizer is aggressive, it may detect that a function's result is unused or that an input is constant, leading it to eliminate the entire computation or to remove it out of a loop, resulting in artificially low (and inaccurate) timing data. The `std::hint::black_box` function acts as an opaque barrier, signaling to the compiler that the value is unknown and must be treated as having side effects, thereby forcing it to execute the code exactly as written within the benchmark. [12] Each benchmark will be run with and without black_box to compare the results and understand the impact of compiler optimizations on our measurements.

Finally, we will compare the results to the benchmark to a baseline implementation that does not use any plugin system, to understand the overhead introduced by each approach compared to a direct implementation of the functionality within the host application. This will allow us to quantify the performance impact of using a plugin system as percentages and to identify any significant differences between the approaches we are evaluating. Using this quantified value, we can then understand how fast each approach is compared to the baseline.

b) *Development complexities:*

When using external crates, they usually come with their own learning curve and documentation. This can add to the development time, especially if the crate is not well-documented or has a complex API. Additionally, integrating external crates into an existing codebase can sometimes lead to compatibility issues or require significant refactoring. Quantifying this is difficult, as it depends on the developer's experience, familiarity with the crate, and the specific requirements of the project. We have decided to instead focus on a qualitative analysis of the development complexities, based on the given available data by the crates maintainers and our experience during the implementation of each approach. This will include factors such as the ease of use of the crate's API, the quality of its documentation, and any challenges faced during integration.

We expect that the dynamic library loading approaches (Rust ABI and C ABI) may have higher development complexities compared to the IPC approach, due to the need to manage dynamic libraries, handle FFI (in the case of C ABI), and ensure compatibility between the plugin and host application. The IPC approach may have lower development complexities in terms of integration, as it allows for more decoupled communication between processes, but it may require more complex software design to handle serialization and inter-process communication effectively.

We will evaluate each approach with a ++, +, 0, - or -- based on the following criteria:

- **++**: The approach is straightforward to implement, with a clear and well-documented API, and requires a low amount of additional setup or configuration. The crate offers good documentation and examples, making it easy for developers to understand and use effectively.
- **+**: The approach is generally manageable to implement, but may require some additional setup or configuration. The crate has decent documentation, but may have some gaps or areas that are not well-explained, which could lead to some challenges during development.
- **0**: The approach is neutral in terms of complexity, with neither significant advantages nor disadvantages in terms of implementation effort or documentation quality.
- **-**: The approach is complex to implement, with a steep learning curve, a poorly documented API, or significant additional setup or configuration required.
- **--**: The approach is very complex to implement, with a very steep learning curve, a poorly documented API, or significant additional setup or configuration required. Alternatively, the approach may have significant disadvantages for the developers, where it feels impossible to use effectively.

c) *Language Limitations:*

Each approach may come with some limitations for the developers use of the Rust language. In general, we want to see how an approach is limiting the use of the existing Rust features, capabilities and ecosystem.

Many of the approaches will refer to the C ABI, which is a common way to achieve a interoperability between Rust and other programming languages, but it also comes with some limitations in terms of the types of data that can be easily represented and the need to use unsafe code to interact with C libraries. Some Rust Types cannot be easily represented in C, which can lead to limitations in the C ABI approach. We also expect that the IPC approach may have limitations in terms of the types of data that can be easily serialized and deserialized, which could impact the flexibility of this approach. Typically, pointers and references are difficult to represent in C, while IPC might even struggle with more complex data structures that require custom serialization logic.

In some cases, Limitations may overlap with Section III.A.b, as certain they may require additional development effort to work around or mitigate. For example, if a particular approach has limitations in terms of the types of data that can be easily represented or serialized, this may require additional development effort to implement custom serialization logic or to design the plugin architecture in a way that avoids these limitations.

Therefore we evaluate each approach with a 0, - or -- based on the following criteria:

- **0**: The approach has no significant limitations for developers, and can be used effectively in a wide range of scenarios without requiring significant workarounds or mitigation strategies.
- **-**: The approach has some limitations that may require developers to implement workarounds or mitigation strategies in order to use it effectively. These limitations may impact the flexibility or usability of the approach, but they are not insurmountable and can be managed with some additional effort.
- **--**: The approach has significant limitations that may make it difficult or impractical for developers to use effectively or potentially even impossible to use in certain scenarios.

d) *Interoperability:*

Interoperability refers to the ability of a plugin system to work seamlessly with other libraries and tools, allowing for easy integration and compatibility with a wide range of use cases. This is an important factor to consider when evaluating different approaches to plugin systems, as it can impact the flexibility and usability of the system in real-world applications.

We perform a qualitative assessment of the interoperability of each approach based on the available documentation.

This is in many cases the core of a plugin system, as it determines how easily plugins can be integrated into the host application and how well they can interact with other libraries and tools. A plugin system with good interoperability will allow developers to easily create and integrate plugins that can work with a wide range of libraries and tools, while a plugin system with poor interoperability may require significant effort to create and integrate plugins, and may limit the functionality of the plugins that can be created.

While an ABI might the solution for the interoperability between an Application to it's plugins, it might not be the best solution for the interoperability between the plugins and other libraries and tools. Typically in cases, where high performance is not required, a communication over IPC might be a better solution for the interoperability between the plugins and other libraries and tools. Using simple data formats like JSON can already achieve the same effect, while typically being easier to implement and maintain than a complex ABI. However, this comes with the trade-off of potentially higher performance overhead due to the need for serialization and deserialization of data, as well as the communication overhead between processes.

In our test case, we only test the interoperability of the plugin system from a Rust Plugin to a Rust Host Application. For the rating, we are still interested in the operability to other languages.

Our rating for each approach will be assigned as ++, +, 0 based on the following criteria:

- **++**: The approach provides seamless interoperability with other libraries and tools, allowing for easy integration and compatibility with a wide range of use cases.

- **+**: The approach has good interoperability with other libraries and tools, but may require some additional configuration or workarounds to achieve full compatibility.
- **0**: The approach has limited interoperability with other libraries and tools, making it difficult to integrate with existing codebases or to use in a wide range of scenarios. In some cases, the approach may be completely incompatible with certain libraries or tools, which can significantly limit its usefulness in real-world applications.

e) *Safety:*

The Rust Language is designed with safety in mind, and it provides strong guarantees against common programming errors such as null pointer dereferences, buffer overflows, and data races. Typically, errors occur through the careless usage of unwinding. [13]

Yet, this does not result in formal verifications, like DO-178C or IEC 61508, which are often required in safety-critical domains. However, the Rust ecosystem has been making strides in this area, with efforts to achieve certifications for certain subsets of the Rust language and libraries, such as the recent achievement of IEC 61508 (SIL 2) certification for a subset of the Rust core library by Ferrous Systems [14], [15].

From this, we can infer that while Rust provides strong safety guarantees, the level of safety can depend on the specific approach used for plugin integration. For example, approaches that involve unsafe code (such as FFI in the C ABI approach) may require more careful handling to ensure safety, while approaches that operate entirely within safe Rust (such as the Rust ABI approach) may provide stronger safety guarantees by default. The IPC approach may also have its own safety considerations, such as ensuring that data is properly validated and that communication channels are secure.

For this thesis, we do not consider it appropriate to perform a quantitative or qualitative analysis of the safety of each approach, as this would require a detailed examination of the specific implementation and usage patterns, which may be beyond the scope of this research.

B. Potential approaches and problems

When developing a plugin system in Rust, there are several potential approaches that can be taken, each with its own set of advantages and disadvantages. In this section we will introduce different crates and give an overview of their functionality. Here, we mainly differentiate between ABI-based approaches Section III.B.a and IPC-based approaches Section III.B.b. We will have a closer look at the chosen approach in the next section Section III.C. The provided list doesn't represent all possible approaches, but rather common crates and methods that are used within the Rust ecosystem for plugin development. We purposefully added some newer crates, which are not as widely used, to give a more comprehensive overview of the available options for plugin development in Rust.

a) *ABI approaches:*

Here, various crates and approaches to utilize the ABI for plugin systems in Rust are introduced, including the unstable Rust ABI, the abi_stable crate, the bindgen crate for C bindings, and the pyo3 crate for Python bindings. Each of these approaches has its own set of advantages and disadvantages in terms of performance, development complexity, limitations, and interoperability. The section also briefly mentions the use of WebAssembly (Wasm) as a potential approach for plugin systems in Rust.

Unstable Rust ABI:

It is possible to load other Rust Libraries as plugins using the currently unstable Rust ABI. However, this approach is complicated by the fact that the Rust ABI is not stable. This means that the layout of data structures, the calling conventions, and other Aspects of the ABI can change between different versions of the Rust compiler, which can lead to compatibility issues when trying to load plugins compiled with different versions of Rust. Additionally, the Rust ABI is not designed for interoperability with other programming languages, which can further complicate the development of a plugin system that needs to interact with libraries or tools written in other languages.

abi_stable crate:

The abi_stable crate enables stable, version-independent interoperability between Rust libraries and executables by generating a custom, C-like ABI that bypasses Rust's unstable internal layout guarantees, allowing dynamically linked shared objects to communicate safely even when compiled with different Rust compiler versions or feature flags. It achieves this through a macro-heavy approach (`#[sabi]`) that enforces strict trait bounds and generates wrapper code to handle memory layout, vtables, and function pointers, effectively creating a "Rust FFI" that is safer than raw extern "C" for complex types. However, this stability comes with significant trade-offs: the generated code introduces non-trivial runtime overhead due to extra indirection and dynamic dispatch, the API is verbose and requires extensive boilerplate that can obscure the underlying logic, and it currently lacks support for many modern Rust features like async/await or generic specialization, making it less suitable for performance-critical paths or projects prioritizing minimal binary size compared to simpler extern "C" or cxx solutions.

rust-bindgen:

The bindgen crate automates the creation of Rust FFI bindings by parsing C or C++ header files using Clang to generate Rust extern blocks and struct definitions that mirror the original C API, effectively eliminating the manual, error-prone process of translating C types into Rust equivalents. It operates by invoking Clang to analyze the header syntax tree, resolving macros and complex type definitions, and outputting a Rust source file that can be integrated into a build script (build.rs) to ensure bindings stay synchronized with header changes. However, this automation introduces significant downsides: the generated code is often verbose, unreadable, and difficult to maintain manually, leading to large binary sizes; it tightly couples the Rust build process to the presence of a compatible Clang installation and the exact C headers, complicating cross-compilation and CI pipelines; and because it exposes the raw C API directly, it frequently forces developers to wrap the generated unsafe code in safe abstractions to prevent memory safety issues, adding an extra layer of development overhead despite the initial automation. [16]

pyo3 (Python bindings):

The pyo3 crate enables seamless bidirectional interoperability between Rust and Python by leveraging the Python C API to expose Rust functions, structs, and enums as native Python objects while allowing Rust code to call Python functions and manipulate Python objects with type safety. It operates through a macro-driven system where the `#[pyclass]` and `#[pymethods]` attributes generate the necessary C bindings and glue code, automatically handling reference counting, memory management, and the conversion of Rust types (like `Vec` or `String`) into their Python equivalents (like `list` or `str`) via the `IntoPy` and `FromPyObject` traits. By utilizing the Global Interpreter Lock (GIL) guard to ensure thread safety during Python interactions and providing a high-level API that abstracts away the complexities of the underlying C API, pyo3 allows developers to write performance-critical Python extensions in Rust that feel and behave like idiomatic Python code, all while maintaining the safety guarantees of the Rust compiler.

Wasm (WebAssembly):

Using WebAssembly with Rust involves compiling Rust code to the `wasm32-unknown-unknown` target, a process streamlined by tools like wasm-pack and wasm-bindgen that bridge the gap between Rust's memory model and JavaScript's environment. By annotating Rust functions with `#[wasm_bindgen]`, developers can automatically generate the necessary glue code to export Rust functions to JavaScript and import JS functions into Rust, handling complex type conversions (such as mapping Rust `Strings` to JS `strings` and `Vecs` to typed `arrays`) and managing memory ownership across the boundary. This workflow typically starts by installing the `wasm32` target via `rustup`, writing Rust logic with the `wasm-bindgen` crate, running `build` to generate a .wasm binary and corresponding JavaScript/TypeScript bindings, and finally loading the resulting module in a browser or Node.js environment to execute high-performance, sandboxed code that interacts seamlessly with the host application.

b) *Interprocess Communication (IPC):*

An alternative to using the ABI for plugin systems in Rust is to use Interprocess Communication (IPC) to allow plugins to communicate with the host application. This approach involves running the plugin as a separate process and communicating with it through a defined protocol, such as JSON-RPC or gRPC. The main advantage of this approach is that it allows for greater flexibility and decoupling between the host application and the plugins, as they can be developed and deployed independently, and can even be written in different programming languages. However, this approach also comes with some disadvantages, such as increased complexity in terms of software design and potential performance overhead due to the need for serialization and deserialization of data, as well as the communication overhead between processes. Additionally, this approach may require more effort to ensure that the communication protocol is robust and secure, especially if sensitive data is being transmitted between the host application and the plugins.

grpc-rust crate:

The grpc-rust crate provides a Rust implementation of the gRPC protocol, allowing developers to create high-performance, language-agnostic RPC (Remote Procedure Call) services that can be used for interprocess communication in plugin systems. By defining service interfaces using Protocol Buffers and generating Rust code with the grpcio-compiler, developers can easily create gRPC servers and clients that communicate over HTTP/2, enabling efficient and scalable communication between the host application and plugins. However, while gRPC offers strong performance and a rich feature set (such as streaming and built-in authentication), it also introduces additional complexity in terms of setup and maintenance, as it requires managing Protocol Buffer definitions, handling serialization/deserialization overhead, and ensuring proper error handling and security measures are in place for interprocess communication. [17]

Additionally, this approach is not specialized for plugin systems and will require further design and implementation work to create a plugin architecture that effectively utilizes gRPC for communication between the host application and plugins. This may involve defining a clear protocol for how plugins should register themselves, how they will be discovered by the host application, and how they will handle requests and responses in a way that is consistent with the overall design of the plugin system.

rustbridge crate:

As stated by the rustbridge crate entry, “rustbridge lets you write shared library plugins in Rust that can be called from Java, Kotlin, C#, Python, Go, Erlang, or another version of Rust — without dealing with the C ABI directly.” [18]

To accomplish this, they wrap a Rust plugin in a `rbp` bundle, which is a zip file containing the plugin’s shared library and a manifest file that describes the plugin’s API and metadata. The rustbridge runtime then loads the `rbp` bundle, reads the manifest to understand the plugin’s API, and uses dynamic loading to call the plugin’s functions from the host application. This approach abstracts away the complexities of dealing with the C ABI directly, allowing developers to write plugins in Rust that can be easily integrated into applications written in various programming languages. However, this approach also introduces some overhead due to the need for dynamic loading and the additional layer of abstraction provided by the rustbridge runtime, which may impact performance compared to more direct approaches like using the unstable Rust ABI or the abi_stable crate.

Communication is done by JSON-RPC, which is a remote procedure call (RPC) protocol encoded in JSON. It allows for communication between a client and a server, where the client can call methods on the server and receive responses. This means that the plugin can be developed in Rust and communicate with the host application using a standardized protocol, which can be easily implemented in various programming languages. However, this also means that there may be some performance overhead due to the need for serialization and deserialization of data, as well as the communication overhead between processes.

In addition, a developer can write the communication protocol by themselves, by using the binary transport layer provided by rustbridge, which allows for more efficient communication between the host application and the plugin, but also requires more effort to implement and maintain compared to using a standardized protocol like JSON-RPC. [19]

C. Selected approach and detailed solutions

When choosing the approach for further analysis, we decided to take a closer look at the **Unstable Rust ABI**, the **abi_stable** crate, the **stabby** crate and on the **rustbridge** crate. We chose those, because they represent a good variety of approaches to plugin systems in Rust, ranging from direct ABI manipulation to IPC-based communication.

The **Unstable Rust ABI** approach is interesting because it represents the most direct way to load Rust plugins, but it also comes with significant challenges due to the instability of the Rust ABI.

To provide stability to the Rust ABI, the **abi_stable crate** generates a custom, C-like ABI that allows for version-independent interoperability between Rust libraries and executables. This approach is interesting because it provides a solution to the instability of the Rust ABI, but it also comes with its own set of trade-offs in terms of performance and complexity.

The creator of the **stabby crate** want to achieve the same goal as the abi_stable crate, but with different goals. The stabby crate generates a custom ABI that is designed to be stable across different versions of Rust, but it does so by using a more lightweight and flexible approach compared to the abi_stable crate. This makes it an interesting alternative to the abi_stable crate, as it may offer better performance and ease of use while still providing stability for plugin systems in Rust.

Finally, we used the **rustbridge crate** as an example for an IPC-based approach to plugin systems in Rust, using JSON-RPC to communicate between processes. Its main focus are plugins written in Rust, which can then be used in various languages, making it especially versatile for interoperability. In addition it also allows to transport data via a binary transport layer, which gives a developer a lot of freedom to design their own communication protocol.

Due to the scope of this thesis, we did not invest time into the WebAssembly (Wasm) approach, as it is different compared to the other approaches, and it would require a significant amount of additional research and implementation effort to properly evaluate it in the context of plugin systems in Rust. WebAssembly (Wasm) is not covered in this paper.

IV. Results

A. Validation of the overall concept

All our plugins have to implement the following trait:

```
trait Fuser {
  fn fuse(&self, data: &[SensorData])
  -> Result<SensorData, Box<dyn std::error::Error>>;
}
```

Listing 1. Fuser trait definition

The `Fuser` trait defines a single method, `fuse(...)`, which takes a slice of `SensorData` and returns a fused `SensorData` or an error.

```
struct SensorData {
  temperature: f64,
  deviation: f64,
}
```

Listing 2. SensorData struct definition

In our benchmark, a `SensorData` represents a tuple of temperature and deviation. From a practical sense, many temperature sensors have a small margin of error (deviation), which can be used to calculate a more accurate temperature reading by fusing the data from multiple sensors.

The implementation of the `fuse(...)` method will be the same for all plugins, as we want to focus on the performance of the different approaches, and not on the implementation of the plugin itself. The only difference will be the way we load and execute the plugin, which will be the focus of our evaluation. In some cases, we needed to make some adjustments to the function parameters and return types to fit the requirements of the different approaches. These differences will be explained in the respective sections.

As a control group, we ran the same benchmark using a static and dynamic variations first, without the use of any Plugin logic, which evaluates to `AverageFuser` and `Box<dyn Fuser>` respectively.

In the following section Section IV.B, we will showcase the results of the test cases for our different approaches, which are the Unstable Rust ABI, the abi_stable crate, the stabby crate and the rust_bridge crate. We will evaluate the performance, development complexity, limitations and interoperability of each approach, and compare them to our control group. Each of those approaches will be evaluated separately and compared to the control group.

B. Description and motivation for the test cases

a) *Control group: Static and Dynamic variation:*

For the control group, we will also evaluate the our criteria, but without the use of any plugin logic, the results will in general refer to the current state of the Rust Language. This is useful, so we can understand how much the different approaches deviate from the current state of Rust, and to understand the overhead introduced by the plugin system.

Performance:

To establish a baseline for the performance of our benchmark, we run it without any plugin logic. This allows us to understand the overhead introduced by the plugin system and to compare the results of the different approaches against this baseline.

TABLE I
Criterion benchmark: Static variation

Test Case	Standard	Blackboxed
Creation	243.18 ps	245.37 ps
Single Fuse	1.2422 ns	2.3027 ns
Multi Fuse	856.15 µs	865.71 µs

TABLE II
Criterion benchmark: Dynamic variation

Test Case	Standard	Blackboxed
Creation	242.86 ps	249.83 ps
Single Fuse	1.2440 ns	2.3058 ns
Multi Fuse	859.60 µs	865.63 µs

When comparing table Table I and table Table II, we can see that the results are very similar, which indicates that the difference between static and dynamic variations does not have a significant impact on the performance of the benchmark. This is expected, as the only difference is how the trait is initialized.

Additionally, there doesn't seem to be a huge impact on the performance when using the `black_box` function, which indicates that the optimizations of the compiler do not have a significant impact on the performance of the benchmark. This is can potentially be due to the targeted architecture of the benchmark.

TABLE III
Gungraun benchmark: Static variation

Test Case	Instructions	Inst. (Blackboxed)	Estimated Cycles	Est. Cyc. (Blackboxed)
Creation	1	1	36	36
Single Fuse	43	89	94	333
Multi Fuse	92	4,500,136	225	15,000,936

TABLE IV
Gungraun benchmark: Dynamic variation

Test Case	Instructions	Inst. (Blackboxed)	Estimated Cycles	Est. Cyc. (Blackboxed)
Creation	1	21	2	140
Single Fuse	99	108	320	371
Multi Fuse	4,500,146	4,500,155	15,001,017	15,001,102

The estimated cycles of the table Table III and table Table IV show that the final results are very similar, regardless of the usage of a static or dynamic variation.

Development complexity:

We are sticking to the standard Rust development process, which is well-documented and widely used in the Rust community. This means that we are using the standard Rust toolchain, including Cargo for package management and build automation, and we are following the standard Rust coding conventions and best practices. This approach allows us to leverage the existing Rust ecosystem and resources, and it provides a familiar development experience for Rust developers.

As this is the control group, this represent the state of rust and therefore we are assigning a rating of **++**.

Limitation:

Our control group does not have any **limitations** on a technical level, as we are not introducing any dynamic loading or plugin logic here. We are using the base library of

Rust and all of the features this language provides. This is standard rust and therefore the limitations are the same as for any other Rust project.

While it is not possible to introduce dynamic plugins in this control group, we can still put this as optimal showcase of minimal limitations that an ideal plugin system might have - therefore we rate this as: **0**.

Interoperability:

In the Rust default usage, **interoperability** is not an option, and therefore we cannot write dynamic plugins. The Rust ABI is not supported by other languages, and therefore we evaluate the control group as follows: **0**.

b) *Unstable Rust ABI:*

Performance:

TABLE V
Criterion benchmark: Unstable Rust ABI variation

Test Case	Standard	Blackboxed
Creation	218.24 µs	217.24 µs
Single Fuse	8.1845 ns	7.4945 ns
Multi Fuse	0.998 ms	1.0009 ms

TABLE VI
Gungraun benchmark: Unstable Rust ABI variation

Test Case	Instructions	Inst. (Blackboxed)	Estimated Cycles	Est. Cyc. (Blackboxed)
Creation	1,090,531	1,090,534	1,637,886	1,637,923
Single Fuse	182	191	553	604
Multi Fuse	10,500,230	10,500,239	21,001,579	21,001,630

We can infer the following from the results of the Criterion benchmark and the Gungraun benchmark for the Unstable Rust ABI approach: Optimizations by the compiler do not result in any performance improvements. This is expected, as the compiler used for the benchmark cannot optimize the code of the plugin, as it is compiled separately and loaded at runtime.

Comparing the results to the control group, we detect a loss of roughly **16%** in performance, which is expected due to the overhead of communication via FFI.

Development complexity:

For the Unstable Rust ABI approach, we are using the standard Rust development process, which is well-documented and widely used in the Rust community. However, due to the unstable nature of the Rust ABI, we need to be careful when updating our rust compiler version. This will essentially lock you in place for the duration of your project. This risk might be okay on a small scale, but as soon as your software or library needs to grow and incorporate other developers or libraries, it is not acceptable to use.

Other complexities that can arise from this approach are:

- You cannot safely pass owned Rust types across the plugin/host boundaries and ensuring that they can be represented in C using `#[repr(C)]`,
- You have to implement an catch-all panic handler for every boundary,
- You have to assign the `#[no_mangle]` attribute to every function that you want to expose to the plugin system,
- You cannot use generics and trait objects across the plugin/host boundaries, as they are fundamentally incompatible with the unstable ABI.

When working with `async` code, a developer needs to be aware, that passing a `Future` type is not possible due to it's opaqueness. This is due to the compiler generating a lot of code for a state machine, which has unknown size and unknown layout. This is getting additionally complicated by the the required management of the asynchronous

runtime (like `tokio`) and usage of `Pin`, which ensures that the memory location of the Future is stable, which is required for the correct execution of the Future.

A developer needs to manage a lot of low-level details when working with the Unstable Rust ABI approach, which is a huge barrier that needs to be overcome to implement a plugin system using this approach. Therefore, we are rating the development complexity of this approach as **--**.

Language Limitation:

In this case, the language limitations are directly interlinked with the development complexity, as the limitations of the Rust language itself are the main reason for the high development complexity of this approach.

With the reasons given above, many features of the rust language are not available when working with the Unstable Rust ABI approach, which significantly limits the capabilities of the plugin system and makes it difficult to implement complex plugins. Therefore, we are rating the language limitations of this approach as **--**.

Interoperability:

There is a very low amount of interoperability with the Unstable Rust ABI approach, as the plugin and the host application must be compiled with the same version of the Rust compiler to ensure compatibility. This means that you cannot use plugins compiled with a different version of Rust, which significantly limits the flexibility and usability of this approach. Other languages are also not able to understand the Rust ABI. While there are some similarities between the C and Rust ABI, the Rust compiler is using its LLVM to optimize parts of the code.

Therefore, we are rating the interoperability of this approach as **0**.

c) *abi_stable (crate):*

Performance:

TABLE VII
Criterion benchmark: Stable Rust ABI variation

Test Case	Standard	Blackboxed
Creation	29.745 ns	29.751 ns
Single Fuse	8.8004 ns	9.2904 ns
Multi Fuse	1.0699 ms	1.0813 ms

TABLE VIII
Gungraun benchmark: Stable Rust ABI variation

Test Case	Instructions	Inst. (Blackboxed)	Estimated Cycles	Est. Cyc. (Blackboxed)
Creation	675,743	675,632	1,130,034	1,129,601
Single Fuse	200	209	740	791
Multi Fuse	10,500,248	10,500,257	21,001,912	21,001,963

Comparing the results to the control group, we detect a loss of roughly **25%** in performance, which is expected due to the overhead of communication via FFI and the additional abstractions introduced by the abi_stable crate to provide a stable ABI for Rust.

Development complexity:

The abi_stable crate provides a stable ABI for Rust, which allows developers to create plugins that can be loaded at runtime without worrying about compatibility issues. However, using this crate requires some additional setup and configuration compared to the standard Rust development process. Developers need to define their plugin interfaces using the abi_stable API, which may require some learning curve and additional effort to understand and use effectively.

The documentation for this is decent, but there are some gaps and areas that are not well-explained, without diving into the source code itself. In addition, the example

implementation shows a very good scenario, but it doesn't explain very well, why certain decision were made in this scenario.

Otherwise, the implementation requires a high amount of boilerplate code, which needs to be studied first before implementing a plugin system using this crate. For example, many common types like `Vec<T>`, `String`, and `Result<T, E>`, need to be converted to their FFI-safe equivalents, like `RVec<T>`, `RString`, and `RResult<T, E>`.

This crate has been used in production by some companies, which indicates that it is a viable option for implementing plugin systems in Rust, but it may not be the best choice for all projects, especially for those that require a high level of performance or have strict requirements for development complexity. Due to it's relatively old age, there are comparatively more resources available online, which can help developers to understand and use this crate effectively.

In total we give this crate rating of **0**, as the initial complexity might be overwhelming, but once you understand the concepts and implement it into your project, it won't introduce additional points of complexity.

Language Limitation:

Outside of the regular limitations of Rust due it's monomorphization at the compile time, the abi_stable crate does not introduce bigger limitations, as it provides a stable ABI for Rust, which allows developers to create plugins that can be loaded at runtime without worrying about compatibility issues.

That being said, the usage of `async` is not possible as this crate doesn't provide a FFI-safe wrapper for the `Future`.

We rate this approach as **0**, as it is common issue that developers face when working with FFI, and it is not specific to the abi_stable crate.

Interoperability:

In general, the abi_stable crate uses the C ABI to provide a stable interface for plugins, which allows for some level of interoperability with other languages that can understand the C ABI. Since the C ABI is very commonly understood by many languages, this allows to still have a good amount of interoperability. Therefore we rate this approach as **+**.

d) *stabby (crate):*

Performance:

TABLE IX
Criterion benchmark: stabby variation

Test Case	Standard	Blackboxed
Creation	81.095 µs	75.139 µs
Single Fuse	7.3371 ns	5.8786ns
Multi Fuse	933.62 µs	907.15 µs

TABLE X
Gungraun benchmark: stabby variation

Test Case	Instructions	Inst. (Blackboxed)	Estimated Cycles	Est. Cyc. (Blackboxed)
Creation	73,983	73,985	133,412	133,423
Single Fuse	90	95	337	380
Multi Fuse	337	380	15,000,970	15,001,013

Comparing the results to the control group, we detect a loss of roughly **9%** in performance, which is expected due to the overhead of communication via FFI and the additional abstractions introduced by the abi_stable crate to provide a stable ABI for Rust.

It is also interesting to see, that here the `black_box` function seem to have a slight performance improvement, which is not expected at first. Taking a look the the 2nd table Table X also doesn't reveal the reason for this. A closer look at the assembly code of the

benchmark or other information of the gungraun benchmark might reveal the reason for this, but this is out of scope for this thesis.

It should also be noted that, at the time of writing, `stabby` version 72.1.1 exhibits a known performance regression on Rust 1.78 and above. All benchmarks for this crate were therefore run on the nightly compiler, which resolves the issue.

Development complexity:

The `stabby` crate offers a relatively clean API for defining ABI-stable plugin interfaces and loading shared libraries at runtime. The macro-driven design keeps the definition of FFI-safe types concise, and the crate ships FFI-safe equivalents of many common standard library types (e.g. `stabby::vec::Vec`, `stabby::string::String`), which reduces the amount of manual conversion code compared to approaches that require writing all wrappers by hand.

The official GitHub repository includes a worked example that serves as a useful starting point. However, it does not cover more advanced scenarios such as error propagation across the boundary, plugin versioning, or panic safety — areas where a developer will need to consult the API documentation or source code directly. The documentation itself is thorough and explains the rationale behind design decisions clearly, which helps when working through less obvious problems.

The main drawback is that `stabby` is a younger crate with a smaller community compared to `abi_stable`. There are fewer third-party tutorials, forum answers, and real-world examples available, which can slow down onboarding. `async` is not currently supported across plugin boundaries, though the crate's documentation describes possible workarounds.

We rate the development complexity of this approach as **0**: the initial learning curve is manageable and the API is well-designed, but the limited community resources and the absence of async support prevent a higher rating.

Language Limitation:

`stabby` does not impose significant restrictions beyond those inherent to any stable-ABI approach. The requirement to use FFI-safe types at plugin boundaries means that some Rust idioms — generic parameters, trait objects using Rust's native vtables, and `async fn` return types — cannot be used directly across the boundary. The crate provides workarounds for each of these cases, though with varying levels of ergonomics. The lack of native async support is the most impactful limitation for projects that rely heavily on asynchronous code. We rate the language limitations of this approach as **0**.

Interoperability:

`stabby` exposes its plugin interface through a C-compatible ABI, which means that host applications or plugins written in other languages with C FFI support (C, C++, Python, etc.) can interoperate with a `stabby`-based plugin system, provided the data layout of shared types is agreed upon. We rate this approach as **+**.

e) *rust_bridge (crate):*

As this crate allows to differentiate between JSON and binary communication, we will evaluate both of these transport layers separately, as they have different performance characteristics and development complexities.

Performance:

Due to an NullHandle error during the plugin creation itself with criterion, we were not able to run the whole criterion benchmark for the rust_bridge crate. It appears that there is some issue when creating the plugin itself, which seems to be related to the reading of the plugin file. Until the end of the deadline for this thesis, a solution was not found.

The benchmark follow the same structure that is required for communicating with the rust_bridge crate. First, we create a request header (json and binary; single and multiple) and then we send it to the plugin, which will process the request by performing the fusion (single or multiple) and return a response. The response has to be additionally parsed in the binary version only. Additionally, the response has the same structure regardless of input size Listing 1.

TABLE XI
Criterion benchmark: rust_bridge (single `SensorData`)

Test Case	JSON	JSON (Blackboxed)	Binary	Binary (Blackboxed)
Request Single	6.9305 ns	9.6154 ns	10.266 ns	10.846ns
Fuse Single	2.7541 µs	7.7651 µs	127.21 ns	127.45 ns
Response	-	-	2.5295ns	6.3095ns
Total (Single)	2.7610 µs	7.7747 µs	139.005 ns	144.605 ns

Looking at the required amount of time for the fusion (which includes the transport of the data), we can clearly see, that this is a heavy process in the JSON version, which is expected due to the overhead of parsing and serializing JSON data. In contrast, the binary version is much faster, which can be attributed to the fact that the binary version does not require parsing and serializing of JSON data. The request header creation is very fast in both versions, which indicates that the communication with the plugin is not affected by the optimizations of the compiler. The response parsing is also very fast in both versions, which indicates that the communication with the plugin is not affected by the optimizations of the compiler.

TABLE XII
Criterion benchmark: rust_bridge (multiple `SensorData`)

Test Case	JSON	JSON (Blackboxed)	Binary	Binary (Blackboxed)
Request Multiple	40.045 ns	965.32 µs	969.30 µs	974.90 µs
Fuse Multiple	460.39 ms	455.40 ms	1.0599 ms	1.0917 ms
Response	-	-	2.5295ns	6.3095ns
Total (Multiple)	460.43 ms	455.40 ms	1.0625 ms	1.0989 ms

When taking a look at the blackboxed values, we can see that the optimizations of the compiler doesn't have a big impact on the results, which indicates that the results are not affected by any optimizations of the compiler. The fusion process is the most time-consuming part of the benchmark, which is expected due to the fact that it involves processing a large amount of data and performing complex calculations. The request header creation in the JSON version seems to be able to have some optimizations.

Comparing the results to the benchmarking results of the rust_bridge crate, we can see that the results are similar, which indicates that the performance of the rust_bridge crate is consistent with the performance of the control group. This is a good indication that the rust_bridge crate is performing as expected and that the results are not affected by any optimizations of the compiler. [19]

TABLE XIII
Gungraun benchmark: Instruction Count for rust_bridge

Test Case	JSON Instructions	JSON Inst. (Blackboxed)	Binary Instructions	Binary Inst. (Blackboxed)
Request Single	43	56	198	202
Request Multiple	92	105	75,382,326	75,382,330
Fuse Single	60,439	60,456	36,793	36,804
Fuse Multiple	4,518,965,517	4,519,030,617	10,536,920	10,536,931
Response	-	-	135	143

TABLE XIV
Gungraun benchmark: Estimated Cycles for rust_bridge

Test Case	JSON Instructions	JSON Inst. (Blackboxed)	Binary Instructions	Binary Inst. (Blackboxed)
Request Single	94	185	459	501
Request Multiple	225	350	155,053,204	155,053,246
Fuse Single	108,962	109,185	61,448	61,584
Fuse Multiple	6,283,037,647	6,283,110,964	21,097,406	21,097,668
Response	-	-	457	581

The instruction count show that the JSON version does require significantly less instructions compared to the binary version, but in the resulting estimated cycles, we can see that the JSON version requires significantly more cycles compared to the binary version, which indicates that the instructions in the JSON version are more complex and require more processing power compared to the binary version. This is expected due to the overhead of parsing and serializing JSON data, which can be computationally expensive.

Here it also confirms that the optimizations of the compiler doesn't have a big impact on the results, which indicates that the results are not affected by any optimizations of the compiler.

Compared to our control group, the JSON transport layer overhead is at around **537%** and for the binary transport layer, the overhead is at around **25%**.

Development complexity:

Overall, the implementation for the rust_bridge crate is rather mixed. While the serialization and deserialization for the JSON data is straightforward and well-documented, the binary communication with the plugin is more complex and requires a deeper understanding of the data layout of your structs. That being said, the example provided for the binary approach can be followed step by step, which makes it relatively easy to implement.

The documentation of the crate is very good with extensive showcases of diagrams and examples. The technical documentation features diagrams for it's architecture, describes the lifecycle of a plugin, how the request and response flow works, and a lot more.

We rate the development complexity of the JSON transport layer as **+** due to it's simplicity, while the binary transport layer is rated as **-** due to the additional complexity of handling binary data and ensuring that the data layout is correct.

Language Limitation:

As stated by the documentation of the crate, the current limitations are related to the reloading of plugins and the usage of multiple instances. [20] Reloading of plugins is technically already possible, but due to it's inherently fragile nature (global state, background threads, third-party libraries side effects) it is currently not recommended to use this feature. Instead it is recommended to restart a process instead of a dynamic reload. For the multiple instances, the crate does support this feature, but the logging and tracing infrastructure is not designed to handle multiple instances, which can lead to some issues when using this feature.

But more importantly is the fact, that the usage of multiple processes doesn't allow the usage of shared memory, which can be a significant limitation for some use cases, as it can limit the performance and scalability of the plugin system. This is especially relevant for use cases that require a high amount of data to be transferred between the host and the plugin, as the overhead of serializing and deserializing data can become a bottleneck.

In total, we rate the limitations of the JSON transport layer and the binary transport layer as **0**. The limitations are there and need to be considered when designing a large and efficient plugin system, but for most other use cases, these limitations can be handled with careful design and implementation.

Interoperability:

A plugin is exported in the form of a rbp-file. Using this file (and the corresponding crates bindings), this plugin can be used in any Rust application, but also other languages like Python, Java, Ruby and more. Due to the serialization to a JSON, it is generally very flexible. The binary layer needs a bit more work, but can be used across the language boundaries as well, as long as the data layout is correct and the corresponding bindings are implemented.

We give this crate a rating of **++** for interoperability, as it allows for a wide range of use cases and can be used in various programming languages, which makes it a very flexible solution for plugin development.

C. Overview and evaluation of the acquired results

Table XV collects the ratings assigned to each approach across all four evaluation criteria. The performance column expresses the overhead relative to the control group's Multi Fuse baseline of 0.856 ms; the remaining columns use the qualitative scale defined in Section III.

TABLE XV
Summary of results.

Crate	Perf. Diff	Development Complexity	Limitation	Interoperability
native		++	0	0
unstable_abi	16%	--	--	0
abi_stable	25%	0	0	+
stabby	9%	0	0	+
rust_bridge (JSON)	537%	+	0	++
rust_bridge (Binary)	25%	-	0	++

a) *Performance:*

The stabby crate performed the fastest among the stable-ABI approaches, with an overhead of roughly 9% compared to the control group. The abi_stable crate had the highest overhead among the in-process approaches at roughly 25%.

The out-of-process approach using the rust_bridge crate showed a stark contrast between its JSON and binary transport layers. The JSON transport layer's overhead ballooned to roughly 537% for large payloads, while the binary transport layer maintained an overhead close to 25%, comparable to the in-process approaches, even at scale.

b) *Development complexity:*

The **Unstable Rust ABI** is rated **--** and stands apart from every other approach. It demands manual `#[no_mangle]` annotations, `#[repr(C)]` type definitions, a catch-all panic handler at every boundary, and a fixed compiler version across the entire project for the lifetime of the plugin system. These constraints accumulate into a significant maintenance burden that disqualifies the approach for any project expected to grow or to incorporate third-party libraries.

The remaining approaches are more practical. **abi_stable** and **stabby** both receive a **0**: they replace the Unstable ABI's manual busywork with a macro-driven type system, at the cost of an initial learning curve and the requirement to use FFI-safe type wrappers. Between the two, `stabby`'s API is more modern and its documentation more explicit about its design rationale, but its smaller community means fewer secondary resources. **rustbridge**'s JSON transport layer receives **+** for being the most accessible of all approaches — developers work entirely in safe Rust with standard serializable types — while the binary transport layer is rated **-** because correctly specifying the binary layout of shared structs requires careful attention and is easy to get wrong silently.

c) *Language limitations:*

All approaches except the Unstable Rust ABI are rated **0**. The shared limitation across `abi_stable`, `stabby`, and `rustbridge` is the absence of native `async` support

across plugin boundaries. For `abi_stable` and `stabby` this is a consequence of the Rust compiler generating opaque, unsized state machines for `async fn` return types, which cannot be represented in a C-compatible ABI. For `rustbridge` it is a consequence of the IPC model: asynchronous code on either side of the boundary works normally, but the plugin invocation itself is a synchronous request/response cycle.

The Unstable Rust ABI is rated **--** because it additionally restricts generics, native trait objects, and any owned Rust type that cannot be expressed as `#[repr(C)]`, effectively reducing the surface area of expressible plugin interfaces to a thin C-like subset of the language.

d) *Interoperability:*

This is the dimension where the approaches diverge most sharply. The **Unstable Rust ABI** receives **0**: even Rust-to-Rust interoperability requires an exact compiler version match, making cross-version or cross-language use impossible in practice.

abi_stable and **stabby** both receive **+**. Their C-compatible ABI surface allows any language with C FFI support to call into a plugin, provided the host understands the data layout. This is a reasonable level of interoperability for a primarily Rust-centric ecosystem.

rustbridge receives **++** for both transport layers. The `rbp` bundle format and JSON-RPC protocol are language-agnostic by design, and the crate already ships official bindings for Java, Kotlin, C#, Python, Go, and Erlang. The binary transport layer requires implementing the binary protocol on the non-Rust side, but the protocol is documented and straightforward. For projects where plugin authors may not be using Rust, or where the host application is a polyglot system, `rustbridge` is the only approach that provides seamless interoperability without requiring the plugin consumer to link against a C library or understand `#[repr(C)]` layouts.

V. Conclusion and Future Work

A. Conclusion

This thesis set out to analyze the available approaches for implementing plugin systems in Rust across four criteria: performance overhead, development complexity, language limitations and interoperability. Four approaches were selected for evaluation — the Unstable Rust ABI, the `abi_stable` crate, the `stabby` crate, and the `rustbridge` crate — and benchmarked against a baseline implementation with no plugin logic.

The results show a clear trade-off spectrum. The **Unstable Rust ABI** approach is the most direct, but its instability effectively locks the entire project to a single compiler version and strips away most of Rust's expressive type system at the plugin boundary, making it unsuitable for any production scenario that expects to evolve over time. The **abi_stable** crate addresses the stability problem at the cost of roughly 25% performance overhead and significant boilerplate, but it is production-proven and reasonably interoperable through its C-compatible interface. The **stabby** crate achieves a similar goal with a more modern, lightweight design and roughly half the performance overhead (9%), but it is a newer project with fewer community resources. The **rustbridge** crate takes a fundamentally different direction by isolating the plugin in a separate process and communicating over IPC; its binary transport layer comes close to the ABI-based approaches in overhead (25%), while its JSON transport layer trades performance (537% overhead) for near-universal language interoperability and the strong isolation guarantee that a crashing plugin cannot take down the host.

No single approach is universally best. Developers who need maximum raw performance within a single-language Rust ecosystem and can tolerate stability risks may prefer the unstable ABI or `stabby`. Developers who need cross-language plugins, or who prioritize crash isolation over throughput, should consider `rustbridge`. Developers who need a battle-tested, broadly compatible solution with acceptable overhead will find `abi_stable` to be the safest bet today.

The broader picture confirms what the community has long argued: the absence of a stable Rust ABI forces developers to choose between fragile native approaches and heavyweight workarounds. According to the 2025 Rust Survey, around 14% of respondents stated that a stable ABI would unblock their use case, and around 24% said it would improve their code. Approximately 13% considered implementing dynamic library plugins to be a significant problem for their productivity [21]. These numbers underline that, while the problem affects a meaningful share of the Rust community, it is not yet treated as a blocker by the language team — leaving plugin developers in a fragmented landscape for the foreseeable future.

B. Future Work

Several directions would extend the findings of this thesis:

- **Extended analysis of development complexity**: A more systematic developer study involving surveys or interviews with practitioners who have implemented Rust plugin systems could provide richer insights into the real-world development challenges and trade-offs beyond what can be inferred from documentation and code analysis alone.
- **Increased benchmark scope**: Expanding the benchmarks to include more complex plugin workloads (e.g., plugins that perform significant computation, manage state, or interact with external resources) would help to understand how the overheads scale in more realistic scenarios.
- **Stabilization of the Rust ABI**: The most impactful change would be an official stable ABI for Rust. RFC #1675 [6] and related proposals have been deferred, but renewed community pressure and growing industrial adoption may eventually make this a priority. A follow-up analysis once any stabilization lands would be valuable.
- **WebAssembly as a plugin runtime**: Wasm was explicitly excluded from this thesis due to scope constraints, but it represents a compelling alternative: it provides a language-agnostic, sandboxed execution environment with a stable binary interface. A

comparative study of Wasm-based plugin systems (e.g., via `wasmtime` or `extism`) against the approaches analyzed here would be a natural next step.

References

[1] A. Inc., “Plug-in Architectures.” Accessed: Apr. 01, 2026. [Online]. Available: https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/LoadingCode/Concepts/Plugins.html

[2] J. Sterne, “plug-in.” Accessed: Apr. 30, 2026. [Online]. Available: https://www.britannica.com/technology/plug-in

[3] B. Editors, “API.” Accessed: Apr. 30, 2026. [Online]. Available: https://www.britannica.com/technology/API

[4] M. O. Manero, “Plugins in Rust: Reducing the Pain with Dependencies.” Accessed: May 03, 2026. [Online]. Available: https://nullderef.com/blog/plugin-abi-stable/

[5] T. libloading Developers, “Crate libloading.” Accessed: May 01, 2026. [Online]. Available: https://docs.rs/libloading/0.9.0/libloading/

[6] iddm, “Make Rust ABI stable enough to provide plugins functionality #1675.” Accessed: Apr. 01, 2026. [Online]. Available: https://github.com/rust-lang/rfcs/issues/1675

[7] steveklabnik, “Define a Rust ABI #600.” Accessed: Apr. 01, 2026. [Online]. Available: https://github.com/rust-lang/rfcs/issues/600

[8] isaac, “A Stable Modular ABI for Rust.” Accessed: Apr. 01, 2026. [Online]. Available: https://internals.rust-lang.org/t/a-stable-modular-abi-for-rust/12347

[9] M. O. Manero, “Plugins in Rust: Getting our Hands Dirty.” Accessed: Apr. 02, 2026. [Online]. Available: https://nullderef.com/blog/plugin-impl/

[10] “Crate criterion.” Accessed: Apr. 06, 2026. [Online]. Available: https://docs.rs/criterion/0.8.2/criterion/

[11] gungraun developers, “Crate gungraun.” Accessed: Apr. 06, 2026. [Online]. Available: https://docs.rs/gungraun/0.17.2/gungraun/

[12] T. R. Foundation, “Function black_box.” Accessed: Apr. 06, 2026. [Online]. Available: https://doc.rust-lang.org/beta/std/hint/fn.black_box.html

[13] T. R. P. Developers, “The Rustonomicon: Unwinding.” Accessed: Feb. 24, 2026. [Online]. Available: https://doc.rust-lang.org/nomicon/unwinding.html#unwinding

[14] P. LeVasseur, “What does it take to ship Rust in safety-critical?.” Accessed: Apr. 14, 2026. [Online]. Available: https://blog.rust-lang.org/2026/01/14/what-does-it-take-to-ship-rust-in-safety-critical/

[15] Ferrocene, “Ferrous Systems Achieves IEC 61508 (SIL 2) Certification for Rust Core Library Subset.” Accessed: Apr. 14, 2026. [Online]. Available: https://ferrous-systems.com/blog/ferrocene-libcore-news-release/

[16] T. bindgen Developers, “Crate bindgen.” Accessed: May 01, 2026. [Online]. Available: https://docs.rs/bindgen/0.72.1/bindgen/

[17] T. grpc-rust Developers, “grpc-rust.” Accessed: May 01, 2026. [Online]. Available: https://crates.io/crates/grpc

[18] T. rustbridge Developers, “rustbridge v1.0.1.” Accessed: May 01, 2026. [Online]. Available: https://crates.io/crates/rustbridge

[19] T. rustbridge Developers, “Transport Layer.” Accessed: Apr. 30, 2026. [Online]. Available: https://github.com/jrobhoward/rustbridge/blob/main/docs/TRANSPORT.md

[20] T. rustbridge Developers, “Plugin Reload and Multiple Instances.” Accessed: Apr. 30, 2026. [Online]. Available: https://github.com/jrobhoward/rustbridge/blob/main/docs/ARCHITECTURE.md#plugin-reload-and-multiple-instances

[21] T. R. Foundation, “Challenges and wishes about Rust.” Accessed: Apr. 10, 2026. [Online]. Available: https://blog.rust-lang.org/2026/03/02/2025-State-Of-Rust-Survey-results/